SMART SECURITY SYSTEM

A PROJECT REPORT

Submitted by

Krishna Priya Gopalan	BL.SC.U4CSE24224
Rohan Anand	BL.SC.U4CSE24240
Roshne Rajesh	BL.SC.U4CSE24241

for the course

23CSE201- Procedural Programming using C

Guided and Evaluated by

Priya R

Dept. of CSE,



AMRITA SCHOOL OF ENGINEERING, BANGALORE

AMRITA VISHWA VIDHYAPEETHAM

BANGALORE-560 035

November 2025

SMART SECURITY SYSTEM

ABSTRACT:

This project presents an Arduino-based smart security system designed to monitor and safeguard a home environment using multiple Arduino nodes equipped with various sensors and components. Each node performs specific functions such as intruder detection, PIN-based door locking, and card scanning. The system connects wirelessly via Wi-Fi to a web application supported by a Web app, allowing users to monitor and receive real-time alerts. The solution addresses the limitations of traditional home security by providing affordable, integrated, real-time, and user-friendly security with features including access authentication, intrusion detection, alarm activation, and event logging.

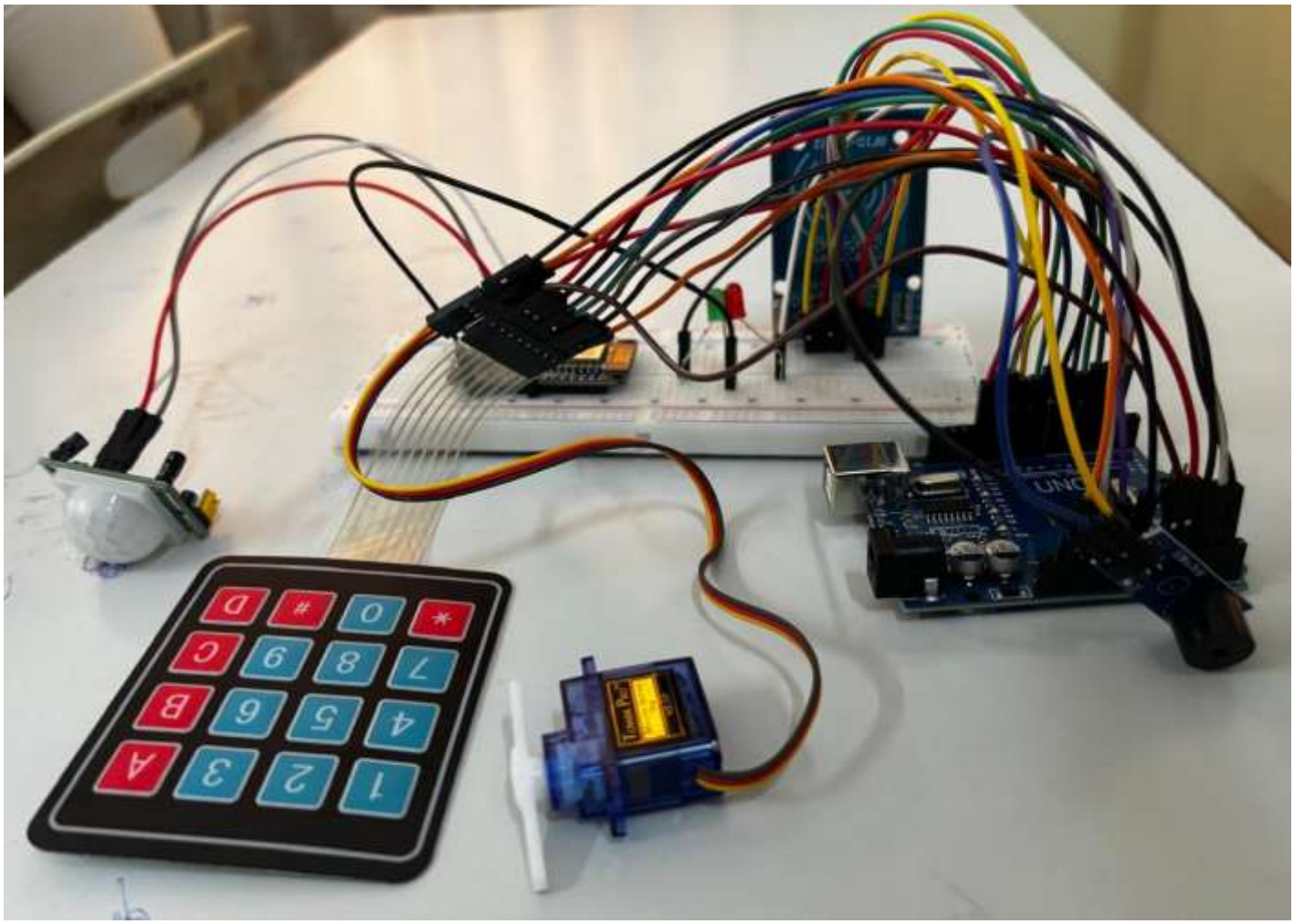
Keywords

Home security, Intruder detection, Wi-Fi connectivity, Real-time warnings, Remote monitoring, Motion detection, Alarm system, User authentication, Security event logging, Centralized monitoring, Affordable security solution, User-friendly interface

1. INTRODUCTION

The increasing need for reliable and accessible home security motivates the development of this Arduino-based smart security system. Unlike conventional security setups that often lack integration, real-time responsiveness, and remote accessibility, this system utilizes multiple Arduino nodes with sensors such as PIR motion detectors, keypad and RFID scanners to detect intrusions and control access. Users can authenticate using PIN codes or cards, while Wi-Fi connectivity enables remote monitoring through a web application. By combining hardware components with a web app, the system ensures timely alerts and detailed event logging to enhance home safety and user convenience.

2. System Model – Diagram



CONNECTIONS:

1. MFRC522 RFID Module:

RFID Pin	Arduino Pin	Function
VCC	3.3V	Power
GND	GND	Ground
RST	D9	Reset Pin
SDA (SS)	D10	Slave Select
SCK	D13	SPI Clock
MOSI	D11	Master Out Slave In
MISO	D12	Master In Slave Out
IRQ	Unconnected	Interrupt

2. 4x4 Keypad:

Keypad Pin	Arduino Pin	Type
Row 1	D2	Digital
Row 2	D3	Digital
Row 3	D4	Digital
Row 4	D5	Digital
Col 1	D6	Digital
Col 2	D7	Digital
Col 3	A1	Analog/Digital

Keypad Pin	Arduino Pin	Type
Col 4	A0	Analog/Digital

3. Servo, Buzzer, and LEDs:

Component	Component Pin	Arduino Pin
Servo Motor	Signal/Data (Yellow wire)	A4
	VCC (Red wire)	5V
	GND (Brown wire)	GND
Buzzer	Signal (Active LOW)	D8
	VCC or GND	5V or GND
Green LED	Anode (+) via 220kΩ Resistor	A2
	Cathode (-)	GND
Red LED	Anode (+) via 220 kΩ Resistor	A3
	Cathode (-)	GND

4. ESP8266 (12E- NODEMCU), PIR Sensor:

Component	Component Pin	ESP8266 Pin (NodeMCU/D-Pin)
PIR Motion Sensor	VCC	3.3V or 5V
	GND	GND
	OUT (Signal)	D5 (GPIO 14)

3. Implementation Details:

a. COMPONENTS USED:

1. **Arduino Uno**: Acts as the main controller. It interfaces with the RFID reader, keypad, servo motor (door lock), LEDs, and buzzer. Handles authentication logic
2. **ESP8266 (ESP-01 12E NODEMCU):** Connects to Wi-Fi and Telegram Bot API. Sends motion detection alerts through Telegram when PIR sensor is triggered.
3. **Hex Keypad (4x4):** Used to enter a PIN for authentication. If correct PIN is entered, the servo unlocks the door.
4. **PIR Sensor**: Detects human motion. When motion is detected, ESP8266 triggers a Telegram notification.
5. **Bread Board**: Used for circuit prototyping and interconnecting all electronic components.
6. **Servo Motor**: Acts as a door lock mechanism. Rotates 0° to lock and 180° to unlock.
7. **Buzzer (Active Low):** Provides audio feedback — short beep for success and multiple beeps for failure.
8. **Resistors (2.2 kΩ):** Used for current limiting, especially for LEDs and to protect microcontroller pins.
9. **Led Lights:** Indicate authentication status
    a. Red = Access Denied
    b. Green = Access Granted
10. **RFID Scanner:** Reads RFID card/tag UID to verify against the authorized card ID stored in the Arduino.
11. **Valid card / Invalid card:** Valid card grants access; Invalid card triggers access denied sequence.

b. FUNCTIONS USED:

I. Arduino Uno Code:

1. setup ()

- Initializes serial communication, SPI bus, RFID module, servo, buzzer, LEDs, and keypad.
- Prints system startup information.
- Positions servo at "lock" state initially.

2. loop ()

- Continuously checks for new RFID cards or keypad inputs.
- If RFID card detected:
  - Reads UID.
  - Compares with authorized card ID.
  - If valid → unlocks door.
  - If invalid → activates failure signal.
- If no card detected → checks for keypad inputs.

3. checkKeypad ()

- Reads key pressed on the keypad.
- Builds the entered PIN.
- If # pressed → checks entered PIN against the stored PIN.
  - If correct → unlocks door (servo rotates 180°).
  - If incorrect → access denied (buzzer + red LED).

4. unlockServo ()

- Rotates servo motor to 180° to unlock door.
- Keeps unlocked for 3 seconds, then relocks (0°).

5. beepBuzzer (unsigned int duration)

- Activates buzzer for a specific duration by pulling buzzer pin LOW (active low).

6. indicateSuccess ()

- One short buzzer beep.
- Green LED ON for 1 second (access granted).
- Turns off LED afterward**.**

7. indicateFailure ()

- Three short buzzer beeps.
- Red LED ON for 2 seconds (access denied).
- Turns off LED afterward.

II. ESP8266 (NodeMCU) Code:

1. setup ():

- Connects to Wi-Fi using SSID and password.
- Initializes Telegram Bot with token and chat ID.
- Configures the PIR sensor and sets an interrupt for motion detection.
- Sends a “Bot started up” message to the Telegram chat on successful boot.

2. loop ():

- Continuously checks if motion is detected.
- If motion detected:
  - Sends “Motion Detected!!” message to Telegram.
  - Waits 10 seconds before next detection to prevent spamming.

3. detectsMovement ():

- Interrupt Service Routine (ISR).
- Triggered when the PIR sensor output goes HIGH.
- Sets motionDetected = true; to notify main loop**.**

4. bot.sendMessage (CHAT_ID, "Message", ""):

- Sends a message to a Telegram chat using the BOT token and chat ID.
- Used to notify the user of system startup or motion detection**.**

5. WiFi.begin(ssid, password)

- Connects the NodeMCU to a local Wi-Fi network.

4. Code (Complete code)

a. Arduino:

```
#include <SPI.h>
#include <MFRC522.h>
#include <Keypad.h>
#include <Servo.h>

// RFID pins
#define SS_PIN 10
#define RST_PIN 9
MFRC522 mfrc522(SS_PIN, RST_PIN);

// Keypad setup
const byte ROWS = 4;
const byte COLS = 4;
char hexaKeys[ROWS][COLS] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'}
};
byte rowPins[ROWS] = {2, 3, 4, 5};
byte colPins[COLS] = {6, 7, A1, A0};
Keypad customKeypad = Keypad(makeKeymap(hexaKeys), rowPins, colPins,
ROWS, COLS);

// Servo setup
Servo myservo;

// Buzzer pin (active low)
#define BUZZER_PIN 8

// LED pins
#define GREEN_LED A2
#define RED_LED A3

// Authentication details - ONLY ONE AUTHORIZED CARD
String correct_rfid = "CD24E321"; // ONLY this card is authorized
String correctPin = "1234";
String enteredPin = "";

void setup() {
```

```
  Serial.begin(9600);
  while (!Serial);
  delay(100);

  Serial.println("Initializing SPI...");
  SPI.begin();

  Serial.println("Initializing RFID...");
  mfrc522.PCD_Init();
  delay(100);

  mfrc522.PCD_DumpVersionToSerial();
  Serial.println("RFID Reader Version:");

  myservo.attach(A4);
  myservo.write(0); // Lock position initially

  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, HIGH); // Buzzer off initially

  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  digitalWrite(GREEN_LED, LOW);
  digitalWrite(RED_LED, LOW);

  Serial.println("=====================================");
  Serial.println("    SMART SECURITY ACCESS SYSTEM   ");
  Serial.println("=====================================");
  Serial.println("System Ready!");
  Serial.println("Authorized Card: " + correct_rfid);
  Serial.println();
  Serial.println(">>> AUTHENTICATION OPTIONS <<<");
  Serial.println("1. Scan your RFID card near the reader");
  Serial.println("2. Enter 4-digit PIN using keypad + press '#'");
  Serial.println();
  Serial.println("NOTE: Press '*' to clear PIN entry");
  Serial.println("=====================================");
  Serial.println("Waiting for authentication...");
  Serial.println();
}

void loop() {
  if (!mfrc522.PICC_IsNewCardPresent()) {
```

```
  delay(50);
  checkKeypad();
  return;
 }

 if (!mfrc522.PICC_ReadCardSerial()) {
  Serial.println("Failed to read card");
  delay(100);
  return;
 }

 Serial.println("\n--- CARD DETECTED ---");

 String rfidUIDSpaced = "";
 String rfidUIDNoSpace = "";

 for (byte i = 0; i < mfrc522.uid.size; i++) {
  if (mfrc522.uid.uidByte[i] < 0x10) {
   rfidUIDSpaced += "0";
   rfidUIDNoSpace += "0";
  }
  rfidUIDSpaced += String(mfrc522.uid.uidByte[i], HEX);
  rfidUIDNoSpace += String(mfrc522.uid.uidByte[i], HEX);

  if (i < mfrc522.uid.size - 1) {
   rfidUIDSpaced += " ";
  }
 }

 rfidUIDSpaced.toUpperCase();
 rfidUIDNoSpace.toUpperCase();

 Serial.print("Scanned UID: ");
 Serial.println(rfidUIDSpaced);

 // Check if card matches the ONE authorized card
 if (rfidUIDNoSpace == correct_rfid) {
  Serial.println("✓ AUTHORIZED CARD!");
  indicateSuccess();
  unlockServo();
  delay(3000); // Keep unlocked for 3 seconds
  myservo.write(0);
  Serial.println("RFID Success: UNLOCKED then LOCKED");
```

```
  } else {
    Serial.println("✗ UNAUTHORIZED CARD - ACCESS DENIED!");
    indicateFailure();
  }

  Serial.println("--- END SCAN ---\n");

  mfrc522.PICC_HaltA();
  mfrc522.PCD_StopCrypto1();

  delay(1000);
}

void checkKeypad() {
  char key = customKeypad.getKey();
  if (key) {
    Serial.print("Key pressed: ");
    Serial.println(key);

    if (key == '#') {
      Serial.print("Checking PIN: ");
      Serial.println(enteredPin);

      if (enteredPin == correctPin) {
        Serial.println("✓ PIN CORRECT!");
        indicateSuccess();
        unlockServo();
        delay(3000);
        myservo.write(0);
        Serial.println("PIN Success: UNLOCKED then LOCKED");
      } else {
        Serial.println("✗ WRONG PIN - ACCESS DENIED!");
        indicateFailure();
      }
      enteredPin = "";
    } else if (key >= '0' && key <= '9') {
      enteredPin += key;
      Serial.print("PIN so far: ");
      for (int i = 0; i < enteredPin.length(); i++) {
        Serial.print("*");
      }
      Serial.println();
```

```
      if (enteredPin.length() > 4) {
        enteredPin = enteredPin.substring(enteredPin.length() - 4);
      }
    } else if (key == '*') {
      enteredPin = "";
      Serial.println("PIN cleared");
    }
  }
}

void unlockServo() {
  Serial.println(">>> Unlocking servo...");
  myservo.write(180);
}

void beepBuzzer(unsigned int duration) {
  digitalWrite(BUZZER_PIN, LOW);
  delay(duration);
  digitalWrite(BUZZER_PIN, HIGH);
}

void indicateSuccess() {
  beepBuzzer(200);
  digitalWrite(GREEN_LED, HIGH);
  digitalWrite(RED_LED, LOW);
  delay(1000);
  digitalWrite(GREEN_LED, LOW);
}

void indicateFailure() {
  for (int i = 0; i < 3; i++) {
    beepBuzzer(100);
    delay(100);
  }
  digitalWrite(RED_LED, HIGH);
  digitalWrite(GREEN_LED, LOW);
  delay(2000);
  digitalWrite(RED_LED, LOW);
}
```

b. ESP8266 – 12 E (NODEMCU) :

```
#include <ESP8266WiFi.h>
#include <WiFiClientSecure.h>
#include <UniversalTelegramBot.h>
#include <ArduinoJson.h>

// Replace with your network credentials
const char* ssid = "CRAZY INNIT";
const char* password = "Rohan180308";

// Initialize Telegram BOT
#define BOTtoken
"8007475216:AAGO3mAmPHAebFZaGfIAdQ5_rzFTd1rQfWg"

// Make sure CHAT_ID has no extra spaces
#define CHAT_ID "8265173011"

X509List cert(TELEGRAM_CERTIFICATE_ROOT);
WiFiClientSecure client;
UniversalTelegramBot bot(BOTtoken, client);

const int motionSensor = 14; // PIR Motion Sensor
volatile bool motionDetected = false;

// Interrupt service routine for motion detection
void ICACHE_RAM_ATTR detectsMovement() {
  motionDetected = true;
}

void setup() {
  Serial.begin(115200);
  configTime(0, 0, "pool.ntp.org"); // get UTC time via NTP

  client.setTrustAnchors(&cert); // Add root certificate for api.telegram.org

  pinMode(motionSensor, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(motionSensor), detectsMovement,
RISING);

  Serial.print("Connecting Wifi: ");
  Serial.println(ssid);
```

```
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    Serial.print(".");
    delay(100);
  }

  Serial.println("");
  Serial.println("WiFi connected");
  Serial.print("IP address: ");
  Serial.println(WiFi.localIP());

  bool started = bot.sendMessage(CHAT_ID, "Bot started up", "");
  Serial.println(started ? "Startup message sent" : "Failed to send startup
message");
}

void loop() {
  if (motionDetected) {
    bool sent = bot.sendMessage(CHAT_ID, "Motion detected!!", "");
    Serial.println(sent ? "Motion message sent" : "Failed to send message");
    Serial.println("Motion Detected");
    motionDetected = false;
    delay(10000); // Wait 10 seconds before allowing another motion message
  }
}
```

5. Output:

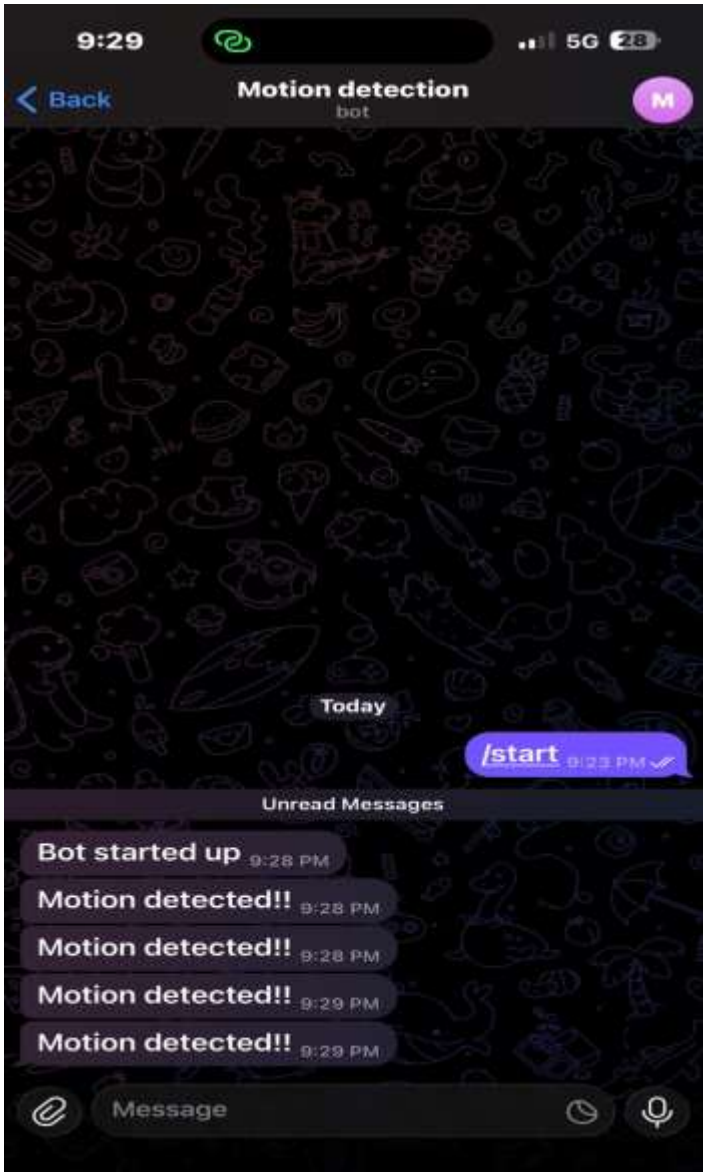
a. Serial monitor output of RFID scanner

```
09:08:02.351 -> Initializing SPI...
09:08:02.351 -> Initializing RFID...
09:08:02.490 -> Firmware Version: 0x92 = v2.0
09:08:02.490 -> RFID Reader Version:
09:08:02.490 -> ====================================
09:08:02.538 -> System Ready!
09:08:02.584 -> Authorized Card: CD24E321
09:08:02.631 -> Please scan your RFID card...
09:08:02.631 -> ====================================
09:08:36.754 ->
09:08:36.754 -> --- CARD DETECTED ---
09:08:36.754 -> Scanned UID: CD 24 E3 21
09:08:36.801 -> ✓ AUTHORIZED CARD!
09:08:37.925 -> >>> Unlocking servo...
09:08:40.956 -> RFID Success: UNLOCKED then LOCKED
09:08:40.956 -> --- END SCAN ---
09:08:40.956 ->
09:08:42.873 ->
09:08:42.873 -> --- CARD DETECTED ---
09:08:42.873 -> Scanned UID: 6D 80 F6 21
09:08:42.920 -> ✗ UNAUTHORIZED CARD - ACCESS DENIED!
09:08:45.526 -> --- END SCAN ---
09:08:45.526 ->
```

b. Serial Monitor output for Hex Keypad:

```
09:13:06.000 -> System Ready!
09:13:06.000 -> Authorized Card: CD24E321
09:13:06.047 ->
09:13:06.047 -> >>> AUTHENTICATION OPTIONS <<<
09:13:06.094 -> 1. Scan your RFID card near the reader
09:13:06.094 -> 2. Enter 4-digit PIN using keypad + press '#'
09:13:06.188 ->
09:13:06.188 -> NOTE: Press '*' to clear PIN entry
09:13:06.188 -> ==================================
09:13:06.235 -> Waiting for authentication...
09:13:06.281 ->
09:13:22.985 -> Key pressed: 1
09:13:22.985 -> PIN so far: *
09:13:40.782 -> Key pressed: 2
09:13:40.782 -> PIN so far: **
09:13:41.469 -> Key pressed: 3
09:13:41.469 -> PIN so far: ***
09:13:42.572 -> Key pressed: 4
09:13:42.572 -> PIN so far: ****
09:13:43.778 -> Key pressed: #
09:13:43.778 -> Checking PIN: 1234
09:13:43.778 -> ✓ PIN CORRECT!
09:13:44.978 -> >>> Unlocking servo...
09:13:47.979 -> PIN Success: UNLOCKED then LOCKED
09:13:50.098 -> Key pressed: 8
09:13:50.098 -> PIN so far: *
09:13:50.385 -> Key pressed: 8
09:13:50.385 -> PIN so far: **
09:13:50.714 -> Key pressed: 8
09:13:50.714 -> PIN so far: ***
09:13:51.244 -> Key pressed: 8
09:13:51.244 -> PIN so far: ****
09:13:52.223 -> Key pressed: #
09:13:52.223 -> Checking PIN: 8888
09:13:52.223 -> X WRONG PIN - ACCESS DENIED!
```

c. ESP8266 enabled PIR notification through telegram



6. Key achievements:

- Dual authentication mechanisms using RFID cards and numeric PINs offer robust access control with high security.
- Integrating a Telegram bot allows for real-time notifications, remote monitoring, and immediate alerts to users over motion detection or system status.
- The integrated feedback systems, including audio (buzzer) and visual (LED indicators) cues, provide instant confirmation of authentication success or failure.
- Arduino and ESP8266 platform modular hardware design ensures that scalability for further expansion with more sensors or nodes is possible.
- Cost-effective as opposed to commercial security systems yet still retains essential security features and remote accessibility.

7. Conclusion:

A Smart Security System is an integrated system that provides multiple layers of protection to a house through the use of hardware components with wireless connectivity and real-time alerting mechanisms. This system consists of RFID card scanning, keypad-based PIN authentication, and motion detection, which can be combined to allow flexible access control and intrusion detection.

The implementation successfully addresses traditional security limitations with an affordable, user-friendly, accessible solution that marries hardware reliability with cloud-based real-time monitoring. Machine learning-based anomaly detection, integration with voice assistants, and multi-node mesh networking for extended coverage are some areas of future enhancement. This project forms a very strong foundation for IoT-based home security systems suitable for both residential and small commercial applications.

8. References:

i. D. Sayre, "RFID Applications in Access Control Systems," IEEE Sensors Journal, vol. 15, no. 3, pp. 1245–1253, Mar. 2014.

ii. M. Banáe, L. Musil, and J. Beran, "Internet of Things in Home Security: Sensor Integration and Real-Time Monitoring," IEEE Internet of Things Journal, vol. 8, no. 5, pp. 3892–3905, May 2021.

iii. R. Sharma and P. Gupta, "Arduino-Based Security Systems: Architecture and Implementation," in Proc. 2019 International Conference on IoT and Application Engineering, Bangalore, India, pp. 112–118, Nov. 2019.

iv. S. Kumar, A. Pandey, and V. Singh, "PIR Motion Sensors in Intrusion Detection: Performance Analysis and Optimization," International Journal of Advanced Electronics and Communication Systems, vol. 6, no. 2, pp. 45–62, Feb. 2020.

v. K. Patel, R. Desai, and N. Mehta, "Wi-Fi Enabled IoT Devices for Remote Monitoring: A Comprehensive Survey," IEEE Access, vol. 9, pp. 75234–75251, June 2021.